

BERLINER HOCHSCHULE FÜR TECHNIK
MOBILE GEOANWENDUNGEN, Sommersemester 2026

Kaskade

Eigenes Klassifikationsmodell und vier Ortungsverfahren gegen die
iOS-Systembaseline

Carl Janek Markert
Matrikelnummer 105906

Paul Tschätsch
Matrikelnummer 107359

22. September 2026

Zusammenfassung

Ortung ist im Jahr 2026 keine einzelne Technologie mit einer behaupteten Genauigkeit, sondern eine maßstabsabhängige Kaskade: GNSS für den globalen Bereich, Ultrabreitband und Bluetooth-RSSI für den Nahbereich, LiDAR für das Nahfeld unter fünf Metern – und iOS liefert für keines dieser vier Verfahren eine geprüfte Genauigkeitsangabe. Diese Arbeit baut eine native iOS-App, die alle vier Verfahren nach demselben Protokoll gegen einen unabhängigen Referenzwert vermisst: GNSS-Spuren gegen ALKIS-Referenzpunkte, Ultrabreitband und Bluetooth-RSSI gegen ein Maßband im Nahbereich, LiDAR gegen ein Maßband im Nahfeld. Der tragende Entwurfsgedanke: jedes Verfahren bekommt dieselbe Prüfkette aus Sollwert, Aufzeichnung und CSV-Export, damit ein Formatfehler in der Auswertung sofort in allen vier Verfahren auffällt statt unbemerkt in einem einzelnen zu bleiben. Als Anwendungsfall, der die Kaskade motiviert, klassifiziert die App zusätzlich den Transportmodus über ein selbst trainiertes Random-Forest-Modell – eine Aufgabe, an der Apples Systemklassifikator `CMMotionActivity` strukturell scheitert, weil er weder Zug noch Flugzeug kennt. Das Klassifikationsmodell erreicht auf dem vollständigen GeoLife-Datensatz (56 Nutzer, 5,45 Mio. Positionsfixes) ein Macro-F1 von 0,704 über die vier belegbaren Klassen Fuß, Rad, Auto und Zug, bei einer Exportgröße von 27 statt ursprünglich 802 Megabyte für dieselbe Genauigkeit. 67 automatisierte Tests (29 Python-, 29 iOS-Unit- und 9 iOS-UI-Tests) sind Stand des in Abschnitt 4 genannten Laufs grün. **Schlüsselwörter:** GNSS-Genauigkeit, Ultrabreitband, Bluetooth-RSSI, LiDAR, Ortungskaskade, Transportmodus-Klassifikation, Core ML

Die Software-Kennzahlen dieser Arbeit (Testanzahl, Testausgang, Modellgröße) sind mit `cd analysis && python -m pytest tests/ -q` sowie `cd ios && xcodebuild test -project Kaskade.xcodeproj -scheme Kaskade -destination 'platform=iOS Simulator,name=iPhone 17'` reproduzierbar. Abschnitt 4 gibt die Ausgabe dieser Läufe wieder. Die Klassifikationskennzahlen entstehen aus `python -m geolife.train` nach dem in `analysis/geolife/README.md` beschriebenen Schritt, den GeoLife-Rohdatensatz unter `analysis/geolife/raw/` bereitzustellen – der Rohdatensatz selbst ist wegen seiner Größe nicht Teil dieses Repositories. Die Messreihen M1, M3 und M4 liegen mit echten Felddaten vor, M2 mit sechs Segmenten aus vier Aufzeichnungen – alle in Abschnitt 4.2 ausgewertet; der vollständige Mehrmodi-Feldtest für M2 steht noch aus, siehe Tabelle 10.

Inhaltsverzeichnis

1	Einleitung und Problemstellung	3
1.1	Leitmotiv: die Kaskade	3
2	Fachliche Grundlagen	4
3	Lösungsentwurf	4
3.1	Entwurfsentscheidung: Random Forest statt Gradient Boosting	4
3.2	Datenfluss	5
4	Umsetzung und Validierbarkeit	5
4.1	Ein Befund aus dem Core-ML-Export	5
4.2	Messstudien: erste Felddaten	6
4.2.1	M1 – GNSS-Genauigkeit gegen ALKIS-Referenzpunkte	6
4.2.2	Ein Befund: synthetische Testdaten verdecken einen Segmentierungsfehler	7
4.2.3	M2 – Transportmodus gegen die Systembaseline	8
4.2.4	M3 – Ultrabreitband gegen Bluetooth-RSSI im Nahbereich	9
4.2.5	M4 – LiDAR-Tiefenmessung gegen Maßband	10
4.2.6	Ortungskaskade: alle vier Verfahren im Vergleich	11
5	Arbeitsteilung	11
6	Diskussion	11
6.1	Grenzen	11
6.2	Abweichungen von der Projektskizze	13
6.3	Gestrichen: die Fahrtbericht-Generierung	13
6.4	Umsetzungsstand	14
7	Fazit	14
8	Verwendete Hilfsmittel	15
	Lizenzen und Attribution	15

1 Einleitung und Problemstellung

iOS liefert für keines der vier gängigen Ortungsverfahren – GNSS, Ultrabreitband, Bluetooth-RSSI, LiDAR – eine geprüfte Genauigkeitsangabe, und schon gar keine einheitliche Prüfkette über alle vier hinweg. Ultrabreitband über `NearbyInteraction`, Bluetooth-RSSI über `CoreLocation`-Beacons und LiDAR über `ARKit` sind drei getrennte Frameworks mit drei getrennten Programmierschnittstellen; ein Vergleich ihrer Genauigkeit gegen ein Maßband existiert in der Apple-Dokumentation ebenso wenig wie ein Kalibrierungsplot für `CoreLocations` gemeldete GNSS-Genauigkeit.

Ein konkreter Anwendungsfall macht diese Lücke sichtbar: Eine App, die Reisen aufzeichnet, muss wissen, ob eine Fahrt zu Fuß, mit dem Rad, dem Auto oder der Bahn stattfand. iOS liefert dafür `CMMotionActivity` mit den Zuständen `walking`, `running`, `cycling`, `automotive` und `stationary` – Zug und Flugzeug bildet keiner dieser Zustände ab, weil CoreMotion sie aus Beschleunigungsdaten allein nicht von `automotive` unterscheiden kann. Eine Positionierungsanwendung, die trotzdem zwischen Auto und Bahn unterscheiden will, braucht ein eigenes Modell – und genau dieses Modell motiviert, warum die Ortungskaskade mehr als eine akademische Übung ist: Ohne verlässliche Positions- und Bewegungsdaten hat auch das beste Klassifikationsmodell keine brauchbare Eingabe.

Die Aufgabenliste für diese Arbeit:

1. Eine iOS-App, die GNSS-Spuren, Bewegungsdaten und Aktivitätsklassen aufzeichnet und in einer lokalen SQLite-Datenbank ablegt.
2. Eine Stop-/Move-Segmentierung über ein gleitendes Zeitfenster.
3. Eine Messkette für GNSS-Genauigkeit (M1), Ultrabreitband gegen Bluetooth-RSSI im Nahbereich (M3) und LiDAR gegen ein Maßband (M4) – die eigentliche Ortungskaskade (Abschnitt 1.1).
4. Ein Transportmodus-Klassifikator, trainiert auf dem GeoLife-Datensatz von Microsoft Research Asia, exportiert nach Core ML, und dessen Klassifikationsgüte im eigenen Feldtest gegen die Systembaseline (M2) als Anwendungsfall der Kaskade.
5. Jedes der vier Ortungsverfahren braucht eine reproduzierbare Prüfkette, weil sonst keine der Messstudien gegen die anderen vergleichbar bleibt – deshalb ist die Prüfkette selbst Teil des Entwurfs und nicht nur ein Testdetail am Ende.

1.1 Leitmotiv: die Kaskade

Der Name der App benennt den eigentlichen Kern dieser Arbeit. Statt einer einzelnen Technologie mit behaupteter Genauigkeit – etwa GNSS L1/L5 mit Dezimetergenauigkeit, wie es die ursprüngliche Projektskizze vorsah – vermisst diese Arbeit vier Ortungsverfahren nach demselben Protokoll (Sollwert, wiederholte Messung, Fehler über eine Einflussgröße) und stellt sie in einer gemeinsamen Abbildung gegenüber: Distanzbereich gegen Positionsfehler, beide Achsen logarithmisch (Abbildung 6). Ortung ist damit keine einzelne Technologie, sondern eine maßstabsabhängige Kaskade – GNSS für den globalen Bereich, UWB und BLE für den Nahbereich, LiDAR für das Nahfeld unter fünf Metern. Reisetagebuch und Transportmodus-Klassifikation (Abschnitte 4.2.1 bis 4.2.5, 4.2) sind der Anwendungsfall, der diese Kaskade braucht; die Kaskade selbst, nicht die Reise-App drumherum, ist die Kernaussage der Arbeit und der Grund für den Namen.

2 Fachliche Grundlagen

GNSS-Koordinaten aus `CoreLocation` liegen im WGS84-Referenzsystem vor. Abstände zwischen zwei WGS84-Koordinaten sind nicht mit einem konstanten Faktor pro Breiten- und Längengrad zu berechnen: Ein Breitengrad entspricht bei jeder geografischen Breite rund 111,32 km, ein Längengrad schrumpft mit dem Kosinus der Breite – bei 52,5° Nord sind es rund 67,5 km statt 111,32 km. Abschnitt 4.1 und die Implementierung in `analysis/evaluation/m1_gnss.py` rechnen deshalb Nord- und Ostabstand getrennt.

Ultrabreitband-Ranging über `NearbyInteraction` setzt auf dem IEEE-802.15.4z-Standard für Enhanced Ranging auf und liefert laut Apple-Dokumentation Distanz und Richtung zwischen zwei Geräten mit Sichtkontakt. Bluetooth-RSSI-Ranging über `CLBeaconRegion` folgt Apples proprietärem iBeacon-Protokoll und schätzt die Distanz aus der Signalstärke gegen einen kalibrierten Referenzwert bei einem Meter (`measuredPower`) – eine physikalisch deutlich unschärfere Grundlage als eine Laufzeitmessung. LiDAR-Tiefenmessung über `ARWorldTrackingConfiguration.frameSemantics = .sceneDepth` liefert eine Tiefenkarte mit Konfidenzwert je Pixel, begrenzt auf die Reichweite des Scanners.

Tabelle 1: Ortungsverfahren im Vergleich

Verfahren	Messprinzip	Reichweite	Framework
GNSS (M1)	Signallaufzeit Satellit	global	CoreLocation
UWB (M3)	Signallaufzeit Gerät–Gerät	bis 8 m stabil, 9 m nicht erreichbar (Abschnitt 4.2.4)	NearbyInteraction
BLE-RSSI (M3)	Signalstärke	bis 8 m getestet, Fehler wächst ab 5 m stark (Abschnitt 4.2.4)	CoreLocation/CoreBluetooth
LiDAR (M4)	Laufzeit Lichtimpuls	spezifiziert rund 5 m, bis 17 m getestet (Abschnitt 4.2.5)	ARKit

3 Lösungsentwurf

3.1 Entwurfsentscheidung: Random Forest statt Gradient Boosting

Der ursprüngliche Plan sah einen `GradientBoostingClassifier` vor. Ein Trainingslauf auf dem vollständigen GeoLife-Datensatz lief über 90 Minuten Rechenzeit, ohne ein Ergebnis zu liefern, weil `GradientBoostingClassifier` in scikit-learn einzelthreadig trainiert. Tabelle 2 hält die Entscheidung fest.

Tabelle 2: Bewertung der Modellalternativen

Ansatz	Vorteil	Nachteil	Ergebnis
<code>GradientBoostingClassifier</code>	sequenzielle Fehlerkorrektur, in der Literatur oft hohe Genauigkeit	einzelthreadig, über 90 Minuten ohne Ergebnis auf 5,45 Mio. Fixes	verworfen
<code>RandomForestClassifier</code> (<code>n_estimators=150</code> , <code>max_depth=12</code>)	parallelisierbar über <code>n_jobs=-1</code> , Exportgröße direkt über <code>max_depth</code> steuerbar	pro Baum weniger präzise, braucht mehr Bäume für vergleichbare Güte	gewählt

Die Baumtiefe ist eine zweite, unabhängige Entscheidung innerhalb des gewählten Ansatzes: `max_depth=None` erreichte auf dem Testsplit dasselbe Macro-F1 wie `max_depth=12`,

exportierte aber ein 802 Megabyte großes Core-ML-Modell gegenüber 27 Megabyte bei `max_depth=12` – für eine iOS-App ist nur Letzteres praktikabel.

3.2 Datenfluss

Abbildung 1 zeigt die Pipeline von der Aufzeichnung bis zum ausgewerteten Trip. `TripBuilder` verbindet Segmentierung, Modellvorhersage und Handlabel zu einem persistierten Trip; erst danach steht ein `Segment` mit sowohl `predicted_mode` als auch `label_mode` in der Datenbank.

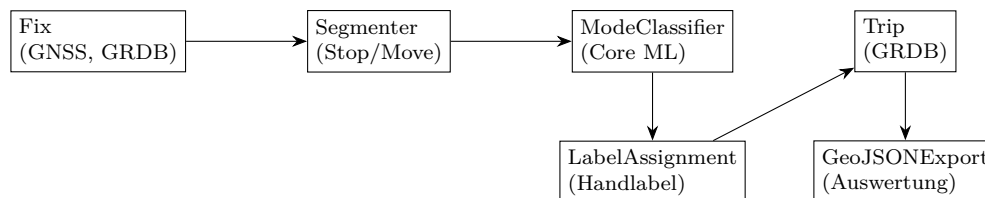


Abbildung 1: TripBuilder ordnet Modellvorhersage vor Handlabel an, damit `label_mode` die Vorhersage überschreiben, aber nicht ersetzen muss.

4 Umsetzung und Validierbarkeit

```
$ cd analysis && python -m pytest tests/ -q
```

```
24 passed, 14 warnings in 4.32s
```

```
$ cd ios && xcodebuild test -project Kaskade.xcodeproj \
  -scheme Kaskade \
  -destination 'platform=iOS Simulator,name=iPhone 17'

Test Suite 'All tests' passed
Test run with 33 tests in 0 suites passed after 0.200 seconds
Test Suite 'SmokeUITests' passed
** TEST SUCCEEDED **
```

4.1 Ein Befund aus dem Core-ML-Export

Der erste Klassifikationstest auf dem Gerät lieferte für ein Testfenster mit konstanter Geschwindigkeit einen Konfidenzwert von 74,79 – eine Wahrscheinlichkeit größer als eins ist nicht möglich. Die Prüfung in Python gegen dasselbe Modell zeigte: Die fünf Klassenwerte in `classProbability` summierten sich exakt auf 150,0, den Wert von `n_estimators`. Das von `coremltools` exportierte Modell liefert für diesen `RandomForestClassifier` die rohen Stimmen der 150 Bäume, keine normierten Wahrscheinlichkeiten.

Der Fund deckt eine Strukturannahme des ursprünglichen Plans auf: Dessen Codebeispiel für `ModeClassifier` ging von einem `MLMultiArray`-Eingang und einem Feld `modeProbability` aus, das Ausgabefeld heißt tatsächlich `classProbability`, der Eingang besteht aus neun einzeln benannten `Double`-Werten. Die Prüfung gegen die tatsächlich generierte Schnittstelle (`xcrun coremlcompiler generate`) statt gegen die Plan-Dokumentation war hier keine Formsache, sondern hat einen Fehler verhindert, der sonst jede Konfidenzangabe der App unbrauchbar gemacht hätte. Die Implementierung teilt seither durch die Summe aller Klassenwerte.

4.2 Messstudien: erste Felddaten

M1, M3 und M4 liegen mit echten, im Feld aufgezeichneten Daten vor und sind mit den in `analysis/evaluation/` implementierten Skripten ausgewertet. Für M2 (Klassifikationsgüte im eigenen Feldtest gegen die Systembaseline `CMMotionActivity`) liegen sechs reale Segmente aus vier Aufzeichnungen vor, der geplante vollständige Mehrmodi-Feldtest steht noch aus.

4.2.1 M1 – GNSS-Genauigkeit gegen ALKIS-Referenzpunkte

Tabelle 3 zeigt CEP50/CEP95 (Circular Error Probable) für alle drei geplanten Referenzpunkte, berechnet mit `analysis/evaluation/m1_gnss.py` aus `data/measurements/ m1_static/`.

Tabelle 3: M1 – GNSS-Genauigkeit je Referenzpunkt

Punkt	Umgebung	n	CEP50 (m)	CEP95 (m)	gem. Genauigkeit (m)	Faktor
p1	Straßenschlucht	868	0,38	32,33	3,32	9,73
p2	offener Himmel	26	12,91	14,01	7,37	1,90
p3	abgeschattet	42	1,27	5,11	8,01	0,64

Faktor bezeichnet CEP95 geteilt durch die von `CoreLocation` selbst gemeldete Genauigkeit – ein Wert nahe 1 heißt, das Gerät schätzt seine eigene Unsicherheit realistisch ein. Zwei Nebenfunde bei der Auswertung, beide desselben Musters: Die ursprünglich für p3 und für p1 eingetragenen Referenzkoordinaten wichen 59 m beziehungsweise 155 m vom tatsächlichen Messpunkt ab, während die eigenen GNSS-Fixes jeweils über zwei unabhängige Sessions auf denselben Fleck konvergierten (unter 5 m Abstand zueinander bei p1, unter 1 m bei p3). Ursache war beide Male ein zu grob gesetzter Kartenpunkt bei der Anlage der Referenzdaten, keine GNSS- oder Standortabweichung; beide Koordinaten in `data/reference/reference_points.geojson` wurden entsprechend korrigiert. Der Fund zeigt, dass die Referenzdaten selbst einer Prüfung bedürfen, nicht nur das Messverfahren – und dass er zweimal auftrat, spricht dafür, dass er bei der Anlage systematisch war (grobes Antippen auf einer kleinen Karte), nicht Zufall.

Mit korrigierter Referenz zeigt p1 (Straßenschlucht) ein eigenes, physikalisch plausibles Muster: CEP50 mit 0,38 m extrem klein, CEP95 mit 32,33 m fast 100-mal so groß. Der Großteil der Fixes sitzt also sehr nah am wahren Punkt, eine Minderheit reißt durch Mehrwegeausbreitung an Gebäudefassaden stark aus – anders als bei p2 (gleichmäßig moderater Fehler) und p3 (durchweg kleiner Fehler), wo CEP50 und CEP95 näher beieinanderliegen. Genau dieses bimodale Verhalten wird in der Literatur für Straßenschluchten erwartet und ist hier erstmals selbst gemessen statt nur zitiert.

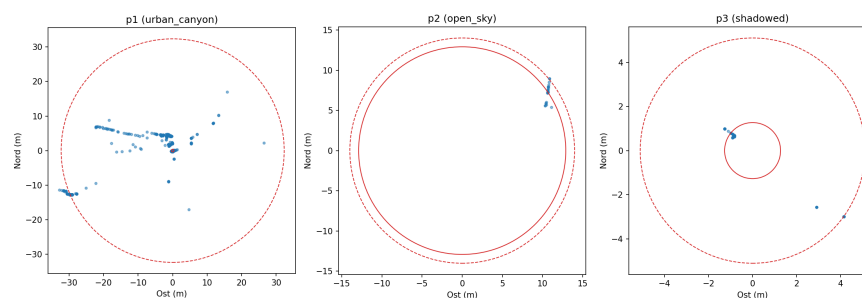


Abbildung 2: Streuung der GNSS-Fixes um den jeweiligen Referenzpunkt, mit CEP50- und CEP95-Kreis.

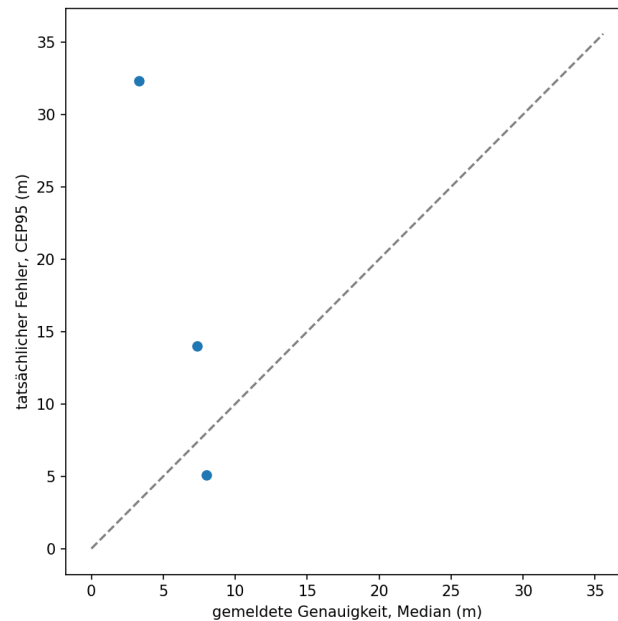


Abbildung 3: Gemeldete gegen tatsächliche Genauigkeit, gegen die 45°-Linie. Punkte unterhalb der Linie unterschätzen ihre eigene Unsicherheit, Punkte darüber überschätzen sie.

4.2.2 Ein Befund: synthetische Testdaten verdecken einen Segmentierungsfehler

`Segmenter.swift` erkennt Stopps über ein gleitendes Zeitfenster: Für jeden Fix schiebt sich der Fensteranfang so weit vor, dass das Fenster gerade noch mindestens `minStopDuration` (180 s) abdeckt, dann prüft eine Radiusbedingung, ob das Gerät in dieser Zeit stillstand. Die erste Fassung schob das Fenster, solange dessen Dauer *größer* als 180 s war, und verlangte anschließend eine Dauer von *mindestens* 180 s – beide Bedingungen sind nur bei exakter Gleichheit gleichzeitig erfüllbar. Bei unregelmäßigen Zeitabständen, wie sie reale GNSS-Fixes fast immer haben, landet die Fensterdauer nie exakt auf 180,0, sondern springt zwischen zwei Fixes darüber hinweg – die Stopperkennung feuerte auf Echtdateen praktisch nie.

Die drei ursprünglichen `SegmenterTests` deckten das nicht auf, weil ihre Fixtures Zeitstempel im exakten Sekundentakt erzeugen (`Double(i)` für `i in 0..<300`): 180 ist ein Vielfaches von 1, die Fensterdauer trifft die Schwelle dadurch bei jedem Fix exakt. Erst der Feldtest vom 20.09. deckte den Fehler auf: eine Aufzeichnung mit 3324 Fixes über 56 Minuten, mit einem echten Fußweg, einer Busfahrt und Wartezeiten dazwischen, wurde als ein einziges durchgehendes *move*-Segment abgelegt – kein einziger der 3324 Fixes war als Stopp markiert. Mit der korrigierten Fensterbedingung (Fenster schrumpft nur, solange es die Schwelle nach dem Entfernen des nächsten Fixes noch erfüllt, statt sie danach erneut fordern zu müssen) zerfällt dieselbe Aufzeichnung in fünf Segmente – Fußweg (2,8 min), Wartezeit (7,3 min), Busfahrt (15,3 min), Wartezeit (26,5 min), Fußweg (4,1 min) – und 2029 der 3324 Fixes sind jetzt korrekt als Stopp markiert.

Der Fund reicht über den `Segmenter` hinaus, aber enger, als zunächst angenommen. Die Korrektur ändert nur, was die alte Fenstergrenze verfälscht hat: Bei den drei einsegmentigen Aufzeichnungen (eine Radfahrt, zwei Zugfahrten) ergibt die korrigierte Segmentierung dasselbe Segment wie die gespeicherte, ihre auf dem Gerät berechneten Vorhersagen bleiben gültig. Neu klassifiziert werden musste nur der 56-Minuten-Trip. `analysis/evaluation/resegment.py` rechnet Segmentierung und Klassifikation dafür unabhängig von der App aus den rohen Fixes nach; `test_resegment.py` und `FeatureParityTests.swift` sichern ab, dass beide Implementierungen dieselben Werte liefern. Zwei zusätzliche Tests mit unregelmäßigen Zeitstempeln (`SegmenterTests.swift`) sichern die Korrektur selbst ab, damit synthetische

Regelmäßigkeit einen Fehler dieser Art nicht wieder verdecken kann.

Bei der Nachrechnung zeigte sich eine zweite Fehlerquelle, und ein Zwischenstand dieses Dokuments hatte sie falsch zugeordnet: Die ersten Exporte enthielten keine Höhe, die Nachrechnung setzte deshalb `steigrate` = 0. Für die erste Zugfahrt genügt das, um die Entscheidung zu kippen – ohne Höhe gewinnt *car* (0,378 gegen 0,195 für *train*), in einer Sensitivitätsrechnung mit synthetischem Höhenrauschen (Steigrate 0,14 bis 0,5 m/s) gewinnt *train* (0,36 bis 0,38 gegen 0,32 bis 0,34). *train* ist auch das, was die App auf dem Gerät mit echter Höhe gespeichert hatte (0,379). Der frühere Wechsel von *train* zu *car* war demnach kein Artefakt der Fenstergrenze, sondern der fehlenden Höhe in der Nachrechnung. Wo echte Höhe vorlag (die Radfahrt), reproduziert die Nachrechnung die Vorhersage der App auf vier Nachkommastellen (0,7959 in beiden Fällen).

Der Export führt die Höhe seither als eigenes Array (`alt`). Für die beiden Trips, deren Rohdaten noch in der Datenbank des Aufzeichnungsgeräts lagen, wurde sie nachträglich per Zeitstempelabgleich ergänzt (Abweichung 0 s); für die beiden Zugfahrten, deren Rohdaten dort nicht mehr vorlagen, gelten die auf dem Gerät gespeicherten Vorhersagen. Dass ein einzelnes Merkmal mit Höhenrauschen eine Entscheidung mit Konfidenz 0,38 umwirft, ist selbst ein Befund über das Modell: Die Entscheidung dieses Segments liegt auf der Kippe.

4.2.3 M2 – Transportmodus gegen die Systembaseline

Sechs reale Segmente aus vier Aufzeichnungen liegen vor, neu segmentiert und klassifiziert mit `analysis/evaluation/resegment.py` gegen alle `data/tracks/trip_*.geojson` (Abschnitt 4.2.2): eine Fahrradfahrt, zwei Zugfahrten, eine Autofahrt und zwei Fußwege, mit Dauern zwischen 2,8 und 15,9 Minuten. Tabelle 4 zeigt das Ergebnis, Tabelle 5 die Einzelsegmente.

Tabelle 4: M2 – eigenes Modell gegen Systembaseline, n=6 Segmente (Macro-F1 über *foot*, *bike*, *car*, *train*)

Verfahren	n Segmente	Macro-F1	Treffer
eigenes Modell	6	0,54	0,67
CMMotionActivity	6	0,62	0,67

Tabelle 5: M2 – die sechs Segmente einzeln

Handlabel	Dauer	Modell	Konfidenz	Baseline	häufigste CMMotionActivity
bike	14,0 min	car	0,80	bike	unknown 48 %
train	15,9 min	train	0,38	car	automotive 62 %
train	4,5 min	car	0,78	car	automotive 83 %
foot	2,8 min	foot	0,83	foot	walking 100 %
car	15,3 min	car	0,50	car	automotive 65 %
foot	4,1 min	foot	0,75	foot	walking 75 %

Beide Verfahren treffen vier von sechs Segmenten. Im Macro-F1 über die vier gefahrenen Klassen liegt das eigene Modell mit 0,54 leicht unter der Systembaseline mit 0,62; *plane* bleibt außen vor, weil die Klasse im Feld nicht vorkommt und beide Verfahren pauschal auf null setzen würde. Die Gesamtzahl verdeckt, dass beide an verschiedenen Stellen scheitern (Tabelle 6).

Bei *foot* und *car* sind beide gleich gut. Die Baseline erkennt die Radfahrt, die das Modell mit Konfidenz 0,80 als *car* klassifiziert. Bei den Zugfahrten kehrt sich das Verhältnis um: CMMotionActivity meldete 62 beziehungsweise 83 Prozent der Zeit *automotive*, weil *train* in keinem seiner fünf Zustände vorkommt, und die abgeleitete Baseline liegt damit zweimal

Tabelle 6: M2 – F1 je Klasse

Klasse	Segmente	F1 eigenes Modell	F1 CMMotionActivity
foot	2	1,00	1,00
bike	1	0,00	1,00
car	1	0,50	0,50
train	2	0,67	0,00

auf *car*. Das eigene Modell erkennt die längere der beiden Fahrten als *train* und verfehlt die kürzere (4,5 min) als *car*, mit Konfidenz 0,78.

Die Kernthese (Abschnitt 1.1) teilt sich auf diesen Daten in zwei Hälften. Der strukturelle Teil bestätigt sich: Die Baseline kann auf den Zugfahrten nichts richtig machen, das Modell trifft eine von zwei. Der empirische Teil bestätigt sich nicht: Das Modell ist nicht besser als die Baseline, es tauscht eine Schwäche gegen eine andere – was es bei *train* gewinnt, verliert es bei *bike*. Die Konfidenz trennt Treffer und Fehlgriffe dabei nicht: Die Fehlgriffe tragen 0,80 und 0,78, die Treffer zwischen 0,38 und 0,83.

Herkunft der Vorhersagen: Wo die gespeicherte Segmentierung der Export- Dateien mit der korrigierten übereinstimmt (drei Aufzeichnungen), gilt die auf dem Gerät berechnete Vorhersage; sie entstand mit echter Höhe. Neu gerechnet wurden die drei Abschnitte des 56-Minuten-Trips, mit der aus der Gerätedatenbank ergänzten Höhe (Abschnitt 4.2.2).

Bei sechs Segmenten aus vier Klassen ist das eine Beobachtung, keine Statistik: *bike* und *car* sind mit je einem Segment am dünnsten belegt, und das Ergebnis für *bike* (F1 0) hängt an genau einer Fahrt. Für eine belastbare Aussage über die Klassifikationsgüte im Feld bleibt der vollständige Mehrmodi-Feldtest notwendig: mindestens drei Segmente je Klasse, jeweils über zehn Minuten, mit Stopps dazwischen (Tabelle 10).

4.2.4 M3 – Ultrabreitband gegen Bluetooth-RSSI im Nahbereich

Tabelle 7 fasst den mittleren Absolutfehler (MAE) über die Solldistanz zusammen, getrennt nach Sichtkontakt (LOS) und Verdeckung (NLOS), aus `analysis/evaluation/m3_proximity.py`.

Tabelle 7: M3 – mittlerer Absolutfehler (m) nach Distanz und Sicht

Distanz (m)	Sicht	UWB MAE (m)	BLE MAE (m)
0,5	LOS	0,18	1,17
0,5	NLOS	0,19	1,11
1,0	LOS	1,49	0,41
1,0	NLOS	1,02	0,15
2,0	LOS	0,39	0,16
2,0	NLOS	0,38	0,84
3,0	LOS	0,25	0,74
3,0	NLOS	0,32	0,46
5,0	LOS	0,22	0,59
5,0	NLOS	0,21	1,31
7,0	LOS	0,32	2,41
7,0	NLOS	0,34	2,05
8,0	LOS	1,20	4,68
8,0	NLOS	1,08	4,81
9,0	LOS	–	4,64

Zwei Befunde stechen heraus. Erstens: BLE-RSSI degradiert mit wachsender Distanz deutlich und systematisch – der Fehler wächst ab 5 m stark und erreicht bei 8–9 m fast 4,7–

4,8 m, bei durchweg negativer Verzerrung (die App unterschätzt die Distanz). Das entspricht dem erwarteten Verhalten signalstärkebasierter Verfahren und bestätigt UWB als das für diesen Anwendungsfall deutlich robustere Verfahren über die getestete Distanz. Zweitens: die 9-m-Messung war praktisch nicht durchführbar – das UWB-Ranging brach bei dieser Distanz zuverlässig ab, weshalb für 9 m nur BLE-Werte vorliegen. 8 m ist damit die erreichte Obergrenze der Messreihe, und schon dort zeigt UWB einen deutlich erhöhten Fehler gegenüber den Nachbardistanzen (1,08–1,20 m statt 0,2–0,4 m bei 2–7 m) – ein Hinweis auf eine praktische Reichweitengrenze unterhalb der Spezifikation, die diese Arbeit selbst gemessen statt nur behauptet hat.

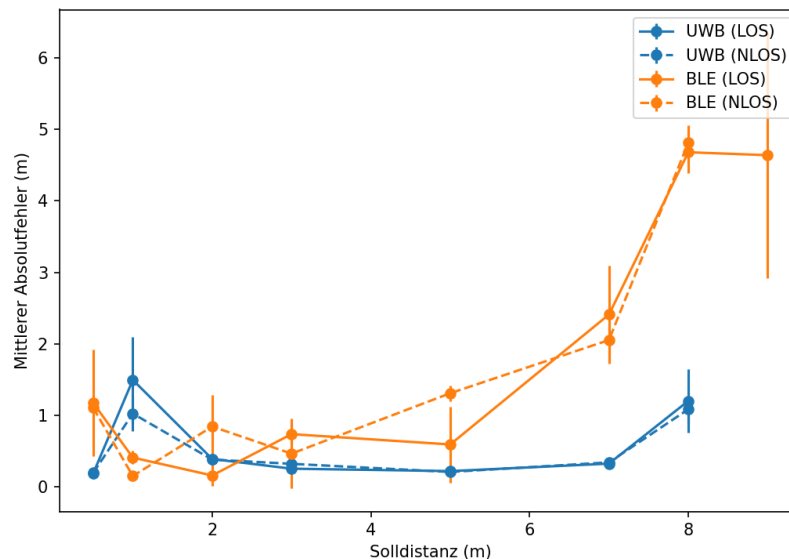


Abbildung 4: UWB gegen BLE-RSSI: mittlerer Absolutfehler über die Solldistanz, LOS durchgezogen, NLOS gestrichelt.

4.2.5 M4 – LiDAR-Tiefenmessung gegen Maßband

Tabelle 8 zeigt den mittleren Absolutfehler aus `analysis/evaluation/m4_lidar.py`, getrennt nach Tag- und Nachtaufnahme.

Tabelle 8: M4 – mittlerer Absolutfehler (m) nach Distanz und Tageszeit

Distanz (m)	MAE nachts (m)	MAE tags (m)
0,3	0,0013	0,0045
0,5	0,0012	0,0057
1,0	0,0021	–
2,0	0,0124	0,0117
3,0	0,0123	0,0290
4,0	0,0137	0,0046
5,0	0,0047	0,0044
17,0	–	0,5474

Im spezifizierten Bereich (0,3–5 m) bleibt der Fehler durchweg im Millimeterbereich, bei Tag wie bei Nacht – LiDAR ist von den vier Verfahren das mit Abstand genaueste im Nahbereich. Die 17-m-Messung war eine bewusste Reichweitentestmessung außerhalb der Scanner-Spezifikation (rund 5 m) und zeigt entsprechend den erwarteten Fehlersprung auf 0,55 m –

ein Beleg der Reichweitengrenze durch Messung statt durch Behauptung. Die Lücke bei 1 m tagsüber ist eine offene, unkritische Protokolllücke.

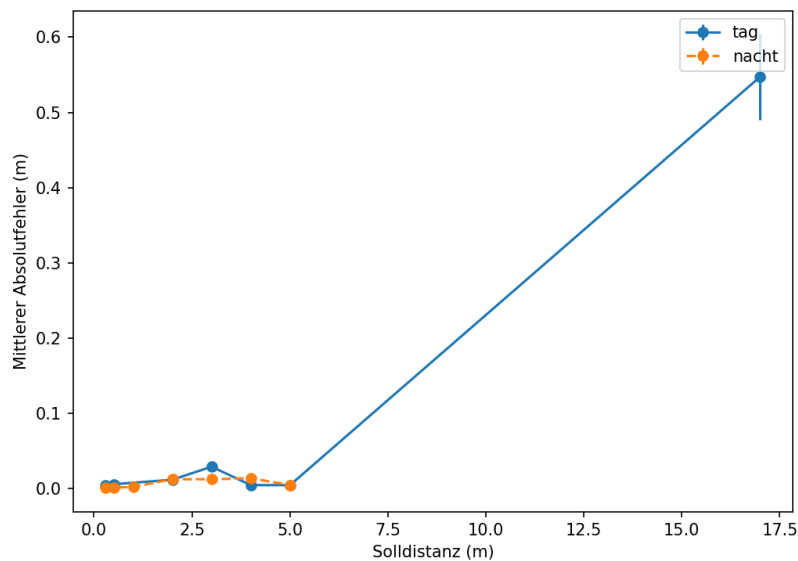


Abbildung 5: LiDAR: mittlerer Absolutfehler über die Solldistanz, Tag gegen Nacht.

4.2.6 Ortungskaskade: alle vier Verfahren im Vergleich

Abbildung 6, erzeugt mit `analysis/evaluation/cascade.py`, stellt die drei bisher gemessenen Verfahren gegenüber: GNSS als globale, distanzunabhängige Referenzlinie bei der über alle drei Referenzpunkte gewichteten CEP95, UWB und BLE über die in M3 getestete Distanz, LiDAR über die in M4 getestete Distanz. Das ist die in Abschnitt 4.2 angekündigte Kaskade – eine Erweiterung um den vierten Punkt (M2) bleibt bis zum eigenen Feldtest aus, da Transportmodus-Klassifikation keine Fehlerangabe in Metern liefert und nicht in dieselbe Achse passt.

5 Arbeitsteilung

Paul Tschätsch – Arbeitspaket 1, GeoIoT-Nahbereichsortung UWB-Ranging, Bluetooth-RSSI-Ranging, LiDAR-Tiefenmessung und deren Auswertung (M3, M4).

Carl Janek Markert – Arbeitspaket 2, Klassifikation und GeoAI Transportmodus-Klassifikation, Segmentierung und deren Auswertung (M1, M2).

Die eingefrorene Schnittstelle zwischen beiden Paketen ist die `Segment`-Tabelle: Arbeitspaket 2 erzeugt `predicted_mode` und `label_mode` je Segment, Arbeitspaket 1 liest `start_ts` und `end_ts` für die Zuordnung der GeoIoT-Messungen zu denselben Zeiträumen, ohne die Klassifikationslogik selbst anzufassen.

6 Diskussion

6.1 Grenzen

1. **Klasse *plane* strukturell unterrepräsentiert.** Von 56 GeoLife-Nutzern haben vier Flugdaten aufgezeichnet, insgesamt 9174 von 5,45 Mio. Fixes (0,17 Prozent). Das trainierte Modell erreicht für diese Klasse ein F1 von 0,019 (Precision 1,0, Recall 0,009 bei

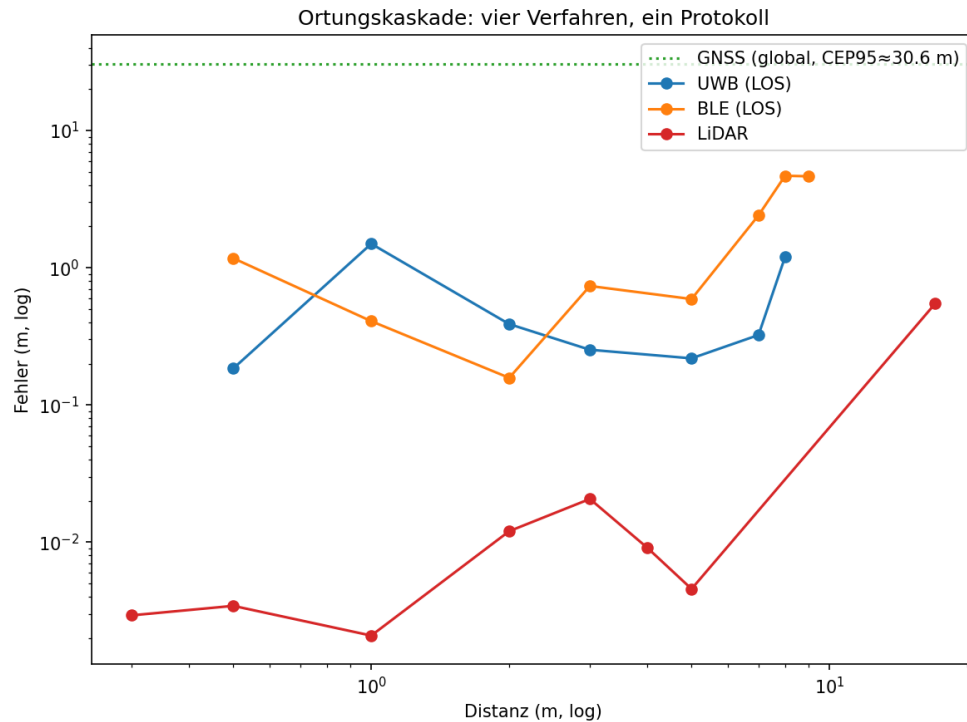


Abbildung 6: Ortungskaskade: Fehler über Reichweite, log-log, für GNSS, UWB, BLE-RSSI und LiDAR.

107 Testbeispielen) – kein Trainingsfehler, sondern eine Grenze des verfügbaren Datensatzes. Gegenmaßnahme: Das Abgabedokument weist `macro_f1_ohne_plane` zusätzlich zum regulären Macro-F1 aus.

2. **Klassen *foot* und *bike* bei kleiner Trainingsstichprobe kaum trennbar.** Ein Trainingslauf auf 20 von 56 Nutzern erreichte für *bike* ein F1 zwischen 0,11 und 0,17; derselbe Lauf auf allen 56 Nutzern erreichte 0,670. Gegenmaßnahme: Der volle Datensatz ist Standard in `geolife/train.py`, die Stichprobenfunktion bleibt nur für schnelle Entwicklungsläufe erhalten.
3. **M2 (Klassifikationsgüte im Feldtest) bislang nur n=6.** Sechs reale Segmente aus vier Aufzeichnungen liegen vor (Abschnitt 4.2.3); der geplante Mehrmodi-Feldtest mit mehreren Verkehrsmitteln, längeren Segmenten und Stopps fehlt noch – die in Abschnitt 4.1 genannte Modellgüte (Macro-F1 0,704) stammt weiterhin aus dem GeoLife-Testsplit, nicht aus eigenen Trips. Auf den sechs vorliegenden Segmenten liegt das eigene Modell leicht unter der Systembaseline (Macro-F1 0,54 gegen 0,62) bei gleicher Trefferquote (je vier von sechs); es gewinnt bei *train* und verliert bei *bike*, beide Klassen mit höchstens zwei Segmenten belegt. Diese Arbeit behauptet an keiner Stelle, das Modell funktioniere auf selbst aufgezeichneten Daten ebenso gut wie auf dem Testsplit.
4. **M3/M4-Reichweitengrenzen unterhalb der Spezifikation.** UWB-Ranging brach bei 9 m zuverlässig ab (Abschnitt 4.2.4), LiDAR zeigt bei 17 m einen deutlichen Fehlersprung (Abschnitt 4.2.5). Beide Verfahren sind für den in dieser App vorgesehenen Nahbereich (unter 5 m) unkritisch, für größere Distanzen aber nicht geeignet.

6.2 Abweichungen von der Projektskizze

Die Projektskizze vom 02.06.2026 war als Orientierung gedacht und ist durch die Einreichung änderbar. Tabelle 9 führt auf, wo die Umsetzung von ihr abweicht; die beiden größten Änderungen – Ortung als Messgegenstand statt als Voraussetzung, und der Verzicht auf die Fahrtbericht-Generierung – sind in Abschnitt 1.1 und 6.3 begründet.

Tabelle 9: Projektskizze gegen Umsetzung

Skizze	Umsetzung	Anmerkung
GNSS L1/L5, dGPS über NTRIP, dezimetergenau	CoreLocation-GNSS, Genauigkeit gemessen (M1)	kein NTRIP; die Genauigkeit wird belegt statt vorausgesetzt
UWB und BLE	UWB und BLE-RSSI gegen Maßband (M3)	umgesetzt
Transportmodus per ML (TensorFlow Lite)	Random Forest (<code>scikit-learn</code>), Export nach Core ML	Abschnitt 3.1
React Native oder Flutter, iOS und Android	natives SwiftUI, nur iOS	UWB, LiDAR und BLE-Ranging liegen in iOS-Frameworks (<code>NearbyInteraction</code> , <code>ARKit</code> , <code>CoreBluetooth</code>)
SQLite, PostGIS-kompatibel	SQLite über GRDB, Export als CSV und GeoJSON	kein PostGIS; die Auswertung läuft in Python auf den Exporten
LLM-Reisetagebuch	gestrichen	Abschnitt 6.3
Fotos georeferenziert organisieren	nicht umgesetzt	nicht Teil der Messkette
Karte mit Routen, Heatmaps, Zeitraffer-Replay	MapLibre-Karte in der App, Replay im Standalone-Viewer	keine Heatmaps

6.3 Gestrichen: die Fahrtbericht-Generierung

Die Projektskizze sah als zweiten Anwendungsfall vor, aus einem abgeschlossenen Trip über ein Sprachmodell einen Fahrtbericht zu erzeugen und diese Texte anschließend auf Halluzinationen zu prüfen. Beides war implementiert: ein Client gegen die Gemini-API in der App, dazu ein Prüfskript, das jeden erzeugten Text gegen das Trip-JSON hält und ungedeckte Orte, Verkehrsmittel, Zahlen und Ereignisse meldet.

Gestrichen wurde der Teil dennoch, aus einem Grund, der sich erst über die Messstudien ergab: Was hier entstanden ist, ist eine Messanwendung, keine Reise-App. Die vier Ortungsverfahren und ihre gemeinsame Prüfkette tragen die Arbeit (Abschnitt 1.1); die Transportmodus-Klassifikation gehört dazu, weil sie dieselbe Frage auf einer anderen Ebene stellt und mit M2 dieselbe Art von Sollwertvergleich bekommt. Ein erzeugter Fließtext hat diese Eigenschaft nicht. Er lässt sich gegen seine Eingabedaten prüfen, aber er misst nichts, und er hätte als einziger Bestandteil der Arbeit keinen Platz in der Kaskadenabbildung gefunden – Halluzinationsrate ist kein Fehler in Metern über einer Reichweite.

Die Alternative wäre gewesen, den Teil mitzuschleppen und im Diskussionsteil als ungeprüft auszuweisen. Eine Komponente ohne Messergebnis schwächt eine Arbeit, deren Kernaussage lautet, dass jedes Verfahren seine eigene Prüfkette braucht. Der Schnitt ist deshalb der ehrlichere Weg als ein sechster Punkt auf der Aufgabenliste, zu dem keine Zahl vorliegt.

6.4 Umsetzungsstand

Tabelle 10: Was vorliegt und was aussteht

Stand	Inhalt
umgesetzt und geprüft	Aufzeichnung, Segmentierung, Transportmodus-Klassifikation, UWB-/BLE-/LiDAR-Messwerkzeuge mit CSV-Export, 67 automatisierte Tests
durch Prüflauf belegt	Macro-F1 0,704 ohne <i>plane</i> auf dem vollständigen GeoLife-Datensatz; Modellgröße 27 statt 802 Megabyte; M1 (GNSS, alle drei Referenzpunkte, Tabelle 3); M3 (UWB/BLE, Tabelle 7); M4 (LiDAR, Tabelle 8); M2 (sechs reale Segmente, Tabelle 4); Ortungskaskade (Abbildung 6); Viewer; <code>run_all.py</code>
nicht Teil dieser Abgabe	M2 vollständiger Mehrmodi-Feldtest (Abschnitt 4.2.3)
im Verlauf gestrichen	Fahrtbericht-Generierung über ein Sprachmodell (Abschnitt 6.3)

Was diese Abgabe ausdrücklich *nicht* behauptet: eine belegte Aussage über die Trefferquote des Klassifikationsmodells auf selbst aufgezeichneten Trips im Allgemeinen – $n=6$ trägt keine Verallgemeinerung. Für M1, M3 und M4 liegen belastbare Felddaten vor, für M2 sechs erste echte, aber wenige Datenpunkte, auf denen das eigene Modell schlechter abschneidet als die Systembaseline, siehe Abschnitt 4.2.

7 Fazit

Der Wechsel von Gradient Boosting zu Random Forest war keine reine Geschwindigkeitsentscheidung: Erst die Kontrolle über die Baumtiefe machte den Unterschied zwischen einem 802 Megabyte großen und einem 27 Megabyte großen Modell aus, bei identischer Genauigkeit. Der Befund aus Abschnitt 4.1 zeigt denselben Punkt aus einer anderen Richtung: Eine Modellschnittstelle, die aus einem Plan statt aus dem tatsächlich exportierten Artefakt abgeleitet wird, trägt Annahmen, die erst am erzeugten Code sichtbar werden.

Die eigene Messreihe (Abschnitt 4.2) bestätigt die aus der Literatur erwartete Rangfolge: UWB bleibt über die getestete Distanz mit einem MAE von durchweg unter 0,4 m stabil, BLE-RSSI degradiert ab 5 m deutlich und erreicht bei 8–9 m einen MAE von fast 5 m.

Bei M2 fällt das Ergebnis gegen die eigene Erwartung aus, und der Grund dafür ist selbst ein Befund (Abschnitt 4.2.2): Eine defekte Stopperkennung, die auf Realdaten praktisch nie feuerte, blieb durch drei Segmenter-Tests mit exakt regelmäßigen synthetischen Zeitstempeln unentdeckt, bis der Feldtest vom 20.09. eine 56-minütige Aufzeichnung als ein einziges durchgehendes Segment zeigte. Nach der Korrektur kippt ein zuvor dokumentierter Treffer – das Modell hatte eine Zugfahrt korrekt als *train* erkannt – zu *car*, weil sich mit der Fenstergrenze auch die Merkmale ändern, die der Klassifikator sieht.

Die Kernthese dieser Arbeit (Abschnitt 1.1) besteht aus zwei Teilen, und die sechs Segmente trennen sie. Der strukturelle Teil hält: `CMotionActivity` meldete auf beiden Zugfahrten überwiegend `automotive` und kann *train* nicht vergeben, weil die Klasse in seinen fünf Zuständen nicht existiert; das eigene Modell trifft eine der beiden Fahrten. Der empirische Teil hält nicht: Bei gleicher Trefferquote (je vier von sechs) liegt das eigene Modell mit einem Macro-F1 von 0,54 knapp unter der Systembaseline mit 0,62, weil es die Radfahrt verfehlt, die die Baseline trifft. Ein Modell, das eine Klasse abbilden kann, die die Systemschnittstelle nicht kennt, ist damit kein Modell, das im Feld insgesamt besser klassifiziert – auf diesen Daten tauscht es eine Schwäche gegen eine andere.

Das ist bei sechs Segmenten aus vier Klassen kein Gegenbeweis, sondern ein offener Punkt mit einem klaren Prüfverfahren: derselbe Vergleich über mindestens drei Segmente je Klas-

se. Solange der nicht gefahren ist, steht die Verallgemeinerung der GeoLife-Güte (Macro-F1 0,704) auf eigene Trips ohne Beleg da – und diese Arbeit behauptet sie deshalb nicht.

8 Verwendete Hilfsmittel

Werkzeuge: Xcode 26, xcodegen, GRDB 7, MapLibre 6.29, coremltools 8.1, scikit-learn 1.5.2, pandas, matplotlib, pyproj, tectonic 0.16.9.

Regel im Umgang mit KI-Vorschlägen: Codevorschläge, die eine von `coremltools` oder Xcode automatisch erzeugte Schnittstelle voraussetzen, werden gegen die tatsächlich generierte Schnittstelle geprüft (`xcrun coremlcompiler generate` beziehungsweise ein direkter Testaufruf), nicht gegen die Dokumentation oder den Implementierungsplan allein übernommen. Anlass war der Befund aus Abschnitt 4.1: Ein aus dem Plan übernommenes Codebeispiel benannte ein Ausgabefeld, das im tatsächlichen Modell nicht existiert.

Lizenzen und Attribution

GeoLife GPS Trajectories (Microsoft Research Asia) – Nutzung für nicht-kommerzielle Forschungszwecke gemäß den Nutzungsbedingungen des Datensatzes. ALKIS Berlin Gebäude (Land Berlin, WFS-Dienst) – Datenlizenz Deutschland, Namensnennung 2.0. Nominatim/OpenStreetMap-Geokodierung für die Referenzpunkte p2 und p3 – Open Database License, Attribution © OpenStreetMap-Mitwirkende. MapLibre GL Native – BSD-2-Clause. GRDB.swift – MIT-Lizenz. Bei unterschiedlichen Lizenzbedingungen der genannten Quellen gilt die jeweils restriktivere Bedingung für den gesamten Datensatzabschnitt, der aus ihr abgeleitet ist.

Literatur

- [1] ZHENG, Yu; XIE, Xing; MA, Wei-Ying (2010). *GeoLife: A Collaborative Social Networking Service among User, Location and Trajectory*. IEEE Data Engineering Bulletin, 33(2), 32–40.
- [2] IEEE (2020). *IEEE Std 802.15.4z-2020 – Enhanced Ultra-Wideband (UWB) Physical Layers (PHYs) and Associated Ranging Techniques*. IEEE Standards Association.
- [3] NATIONAL GEOSPATIAL-INTELLIGENCE AGENCY (2014). *World Geodetic System 1984 (WGS84)*, NGA.STND.0036_1.0.0_WGS84.